

The matchers of the jest:

```
test("null", () => {  
  const i = null;  
  
  expect.assertions(2)  
  
  expect(i).toBeNull();  
  expect(i).toBeDefined();  
});
```

```
const animals = ['Lion', 'Tiger']  
test('array', () => {  
  expect(animals).toContain('Lion')  
})
```

```
test('exception', () => {  
  expect(() => getData()).toThrow('Not Found')  
})
```

```
function getData() {  
  throw new Error('Not Found')  
}
```

To mock the function, we use **jest.mock**, where jest is a global variable. Then we use **mockResolvedValueOnce** to change the value:

```
const crypto = require('node:crypto')
const { getData } = require('../app')
```

```
jest.mock('node:crypto')
```

```
test('Mocking function', async () => {
  crypto.randomBytes.mockResolvedValueOnce('bytes')
  const res = await getData()
  expect(res).toEqual('bytes')
})
```

```
test('Mocking function', async () => {
  crypto.randomBytes.mockImplementationOnce(() => Promise.resolve('bytes'))
  const res = await getData()
  expect(res).toEqual('bytes')
})
```

To use spyOn for mocking with Jest:

```
const crypto = require('node:crypto')
const { getData } = require('../app')
```

```
test('testing spyOn', async () => {
  jest.spyOn(crypto, 'randomBytes').mockImplementationOnce(() => Promise.resolve('JLoka'))

  const t1 = await getData()
  expect(t1).toEqual('JLoka')
})
```

To initialize the configuration file of jest: **npm init jest@latest**

<https://jestjs.io/docs/getting-started#generate-a-basic-configuration-file>

```
jest.mock("../utils/helpers", () => ({
  getJwtToken: jest.fn(() => 'Token@123')
}))
```

```
const mockReq = () => {
  return {
    body: {
      name: "Jafar Loka",
      email: "jloka@jloka.com",
      password: "Test@123"
    }
  }
}
```

```
const mockResp = () => {
  return {
    status: jest.fn().mockReturnThis(),
    json: jest.fn().mockReturnThis()
  }
}
```

```
const mockUserResp = {
  id: 1,
  name: "Jafar Loka",
  email: "jloka@jloka.com",
  password: "Hash@Test@123"
}
```

`jest.restoreAllMocks()`

Restores all mocks and replaced properties back to their original value. Equivalent to calling `.mockRestore()` on every mocked function and `.restore()` on every replaced property. Beware that `jest.restoreAllMocks()` only works for mocks created with `jest.spyOn()` and properties replaced with `jest.replaceProperty()`; other mocks will require you to manually restore them.

To restore the mocks after each test:

```
afterEach(() => {  
  jest.restoreAllMocks()  
  jest.clearAllMocks()  
})
```

To check the rejections and validation with Mongoose:

```
it("should throw validation error", async () => {  
  try{  
    let user = new User()  
  
    // Here we don't call any other 3rd parties methods  
    jest.spyOn(user, 'validate').mockRejectedValueOnce({  
      errors: {  
        name: '',  
        email: '',  
        password: ''  
      }  
    })  
  
    await user.validate()  
  } catch (err) {  
    expect(err.errors.name).toBeDefined()  
    expect(err.errors.email).toBeDefined()  
    expect(err.errors.password).toBeDefined()  
  }  
})
```

To add an error message to the rejection value:

```
it("should throw validation error", async () => {
  try{
    let user = new User()

    // Here we don't call any other 3rd parties methods
    jest.spyOn(user, 'validate').mockRejectedValueOnce({
      errors: {
        password: 'Your password must be at least 8 characters long'
      }
    })

    await user.validate()
  } catch (err) {
    expect(err.errors.password).toBeDefined()
    expect(err.errors.password).toBe('Your password must be at least 8
characters long')
  }
})
```

To test the S3Service:

```
import { S3Service } from './S3Service'; // Adjust path as needed

// 1. Create mock functions outside the jest.mock factory so we can access them
in tests
const mockUploadPromise = jest.fn();
const mockUpload = jest.fn(() => ({
  promise: mockUploadPromise,
}));

-----

// 2. Mock aws-sdk
jest.mock('aws-sdk', () => {
  return {
    S3: jest.fn().mockImplementation(() => ({
      upload: mockUpload,
    })),
  };
});

-----

// 3. Import S3 from aws-sdk just to satisfy imports if needed,
// but we primarily rely on the mocks defined above.
import pkg from 'aws-sdk';
const { S3 } = pkg;

describe('S3Service', () => {
  let s3Service;

  beforeEach(() => {
    // Clear all mock calls and instances
    jest.clearAllMocks();

    // Reset environment variables
    process.env.AWS_ACCESS_KEY_ID = 'test-access-key';
    process.env.AWS_SECRET_KEY = 'test-secret-key';
    process.env.AWS_S3_BUCKET_NAME = 'test-bucket';

    // Reset mock implementations to default success
    mockUploadPromise.mockResolvedValue({
      Location: 'https://test-bucket.s3.amazonaws.com/restaurants/test-file.jpg',
      ETag: '"some-etag"',
    });
  });
});
```

```

    // Instantiate the service
    s3Service = new S3Service();
  });

  -----

  afterEach(() => {
    delete process.env.AWS_ACCESS_KEY_ID;
    delete process.env.AWS_SECRET_KEY;
    delete process.env.AWS_S3_BUCKET_NAME;
  });

  -----

  it('should initialize S3 with correct credentials', () => {
    // Verify S3 constructor was called with correct config
    expect(S3).toHaveBeenCalledWith({
      accessKeyId: 'test-access-key',
      secretAccessKey: 'test-secret-key',
    });
  });

  -----

  it('should upload a file to the correct S3 bucket and path', async () => {
    const mockFile = {
      name: 'test-file.jpg',
      data: Buffer.from('file content'),
    };

    await s3Service.upload(mockFile);

    // Verify upload method was called
    expect(mockUpload).toHaveBeenCalledTimes(1);

    // Verify correct parameters were passed
    expect(mockUpload).toHaveBeenCalledWith({
      Bucket: 'test-bucket/restaurants',
      Key: 'test-file.jpg',
      Body: mockFile.data,
    });
  });
}

```

```

it('should return the upload result', async () => {
  const mockFile = {
    name: 'test-file.jpg',
    data: Buffer.from('file content'),
  };

  const result = await s3Service.upload(mockFile);

  // Verify the promise was called
  expect(mockUploadPromise).toHaveBeenCalled();

  // Verify the result
  expect(result).toEqual({
    Location: 'https://test-bucket.s3.amazonaws.com/restaurants/test-file.jpg',
    ETag: '"some-etag"',
  });
});

```

```

-----

it('should handle upload errors', async () => {
  const mockFile = {
    name: 'test-file.jpg',
    data: Buffer.from('file content'),
  };

  // Simulate an error
  const mockError = new Error('Upload failed');
  mockUploadPromise.mockRejectedValueOnce(mockError);

  // Expect the service method to throw/reject
  await expect(s3Service.upload(mockFile)).rejects.toThrow('Upload failed');
});

```

```

it("should update the job successfully", async () => {

    jest.spyOn(Job, 'findById').mockResolvedValueOnce(mockedJobs[0])
    jest.spyOn(Job,
'findByIdAndUpdate').mockResolvedValueOnce(mockedJobs[0])

    let mockedReq = mockReq()
    mockedReq.params = { id: "64a1b2c3d4e5f6a7b8c9d0e1" }
    mockedReq.body = mockedJobs[0]
    let mockedResp = mockResp()

    await updateJob(mockedReq, mockedResp)
    expect(Job.findByIdAndUpdate).toHaveBeenCalledTimes(1)
    expect(mockedReq.params).toEqual({ id: "64a1b2c3d4e5f6a7b8c9d0e1" })
    expect(mockedReq.body).toEqual(mockedJobs[0])
    expect(mockedResp.status).toBe(200)
    expect(mockedResp.json).toHaveBeenCalledWith({ job: mockedJobs[0] })
})

*****

it("should return job not found", async () => {

    jest.spyOn(Job, 'findById').mockResolvedValueOnce(null)

    let mockedReq = mockReq()
    let mockedResp = mockResp()

    await updateJob(mockedReq, mockedResp)

    expect(mockedResp.status).toBe(404)
    expect(mockedResp.json).toHaveBeenCalledWith({
        error: "Job not found",
    })
})

*****

it("should throw exception", async () => {

    jest.spyOn(Job, 'create').mockRejectedValue({ name: 'ValidationError' })
    let mockedReq = mockReq()
    let mockedRes = mockResp()
    await newJob(mockedReq, mockedRes)
    expect(mockedRes.status).toBe(400)
    expect(mockedRes.json).toHaveBeenCalledWith({
        error: "Please enter all values",
    })
})

*****

```

To test the Mongoose model:

```
test('should be valid with all required fields', () => {
  const job = new Job(getValidData());
  const error = job.validateSync();
  expect(error).toBeUndefined();
});

*****

test('should fail validation if title is missing', () => {
  const data = getValidData();
  delete data.title;
  const job = new Job(data);
  const error = job.validateSync();
  expect(error).toBeDefined();
  expect(error.errors.title.message).toBe("Please enter Job title.");
});

*****

test('should fail validation if industry has invalid enum value', async () => {
  const data = getValidData();
  data.industry = ['Invalid Industry'];
  const job = new Job(data);

  try {
    await job.validate();
  } catch (error) {
    expect(error).toBeDefined();
    expect(error.errors['industry.0'].message).toBe("Please select correct
options for industry.");
  }
});
```